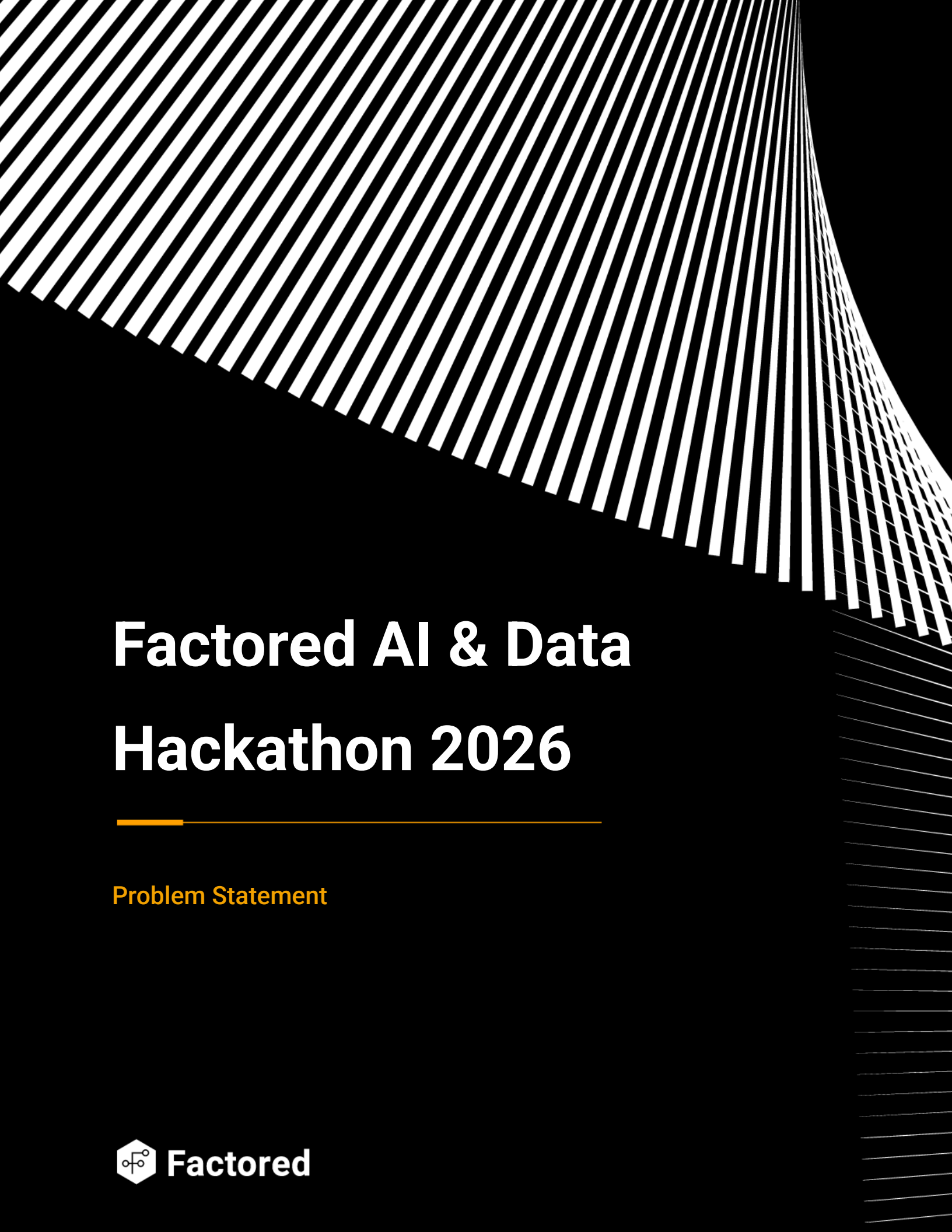




Factored AI & Data Hackathon 2026

Problem Statement

Build an AI-first banking customer service system

Build a working AI-first customer service system for a real-world banking environment.

Your solution should understand complex customer interactions, use data and tools securely, complete appropriate service workflows, and involve human agents when needed.

Choose a focused customer-service problem and demonstrate an end-to-end solution. Use the supplied data to explain why the problem matters, establish a baseline, and measure whether your approach improves service quality and operational efficiency.

Think beyond the demo:

- Design for privacy, explainability, fairness, reliability, and scalability;
- Make explicit trade-offs across autonomy, accuracy, latency, cost, and human oversight;
- Justify where AI is appropriate, where deterministic logic is preferable, and how you evaluate the system for quality and safety.

Scope

Deliver a working prototype with evidence of production readiness and an honest account of the work required before deployment, and your ten-day submission is not expected to operate a live banking service.

Select a coherent workflow, such as account or payment inquiries, card-service support, transaction-dispute intake, or credit-product information and eligibility support.

These are examples, not separate tracks, and depth, demonstrated behavior, and engineering judgment determine the score; implementing more workflows does not earn an automatic bonus.

Include a normal resolution path, an ambiguous or unsupported request, and a case requiring human intervention. Demonstrate interactions in Spanish and Portuguese and report limitations in the supplied data or language coverage.

What your solution should demonstrate

1. A problem supported by data. Analyze contact reasons, relevant demand patterns, data quality, and operational constraints. Use this evidence to prioritize the workflow and define the intended customer and business outcomes.
2. A functioning AI system. Maintain relevant conversational context, clarify ambiguity, and ground factual responses in permitted account, transaction, or policy information. Use tools when they serve the workflow and report only actions whose outcomes the system has verified.
3. Controlled automation. Define which requests the system can answer, which actions require confirmation, and when it must abstain or transfer to a human. Enforce permissions and policy outside model-generated prose. Provide the human agent with the request, verified facts, actions taken, supporting evidence, and unresolved questions.
4. Sound data and ML practice. Build repeatable data preparation with contracts, quality checks, lineage, and an update/freshness policy. Evaluate at least one learned component against an appropriate baseline. Use valid labels or relevance judgments, prevent leakage, and justify representations, metrics, thresholds, and evaluation splits.
5. Measured quality and failure handling. Evaluate on held-out cases. Include incorrect or missing data, expired sessions, unauthorized access attempts,

prompt injection, tool failures, and multilingual ambiguity. Report successful outcomes, unsafe outcomes, handoff behavior, latency, and cost, together with sample sizes and limitations.

6. A credible route to operation. Demonstrate tracing, bounded retries, safe fallback, and reproducible setup. Explain capacity limits, monitoring, access controls, data retention, and the remaining deployment work. Provide explanations based on sources, policy rules, and execution records; hidden model chain-of-thought is not an audit artifact.

Architecture freedom

You may use conventional ML, pretrained language models, retrieval, deterministic workflows, agents, or a justified combination. Training a new model, using multiple agents, reaching a tool-count target, implementing streaming, forecasting demand, and building a dashboard are not mandatory.

Every team is assessed on data engineering and AI/ML rigor. For a pretrained or retrieval-based solution, demonstrate those competencies through component selection, relevance or intent labels, representations, leakage prevention, held-out evaluation, and error analysis. A model-training pipeline is one way to provide that evidence.

Use batch, incremental, or streaming processing according to the supplied inputs and the workflow's latency and freshness needs. Incremental file delivery does not by itself require streaming. If only static data is supplied, demonstrate update correctness with a clearly labeled test fixture.

Data and execution boundaries

Use only organizer-approved data and permitted external resources. Identify which inputs are real, de-identified, synthetic, or team-generated, and follow the published data-use terms. Do not include private customer records, credentials, or restricted data in public submissions or external model requests.

Sandbox services and mock banking tools are acceptable when their contracts and limitations are documented. Demonstrate authentication with a trusted test session or identity service; a national ID or customer number alone does not prove identity. Enforce access to each customer's records and action permissions in the service or tool layer.

For credit-related workflows, separate conversation handling, predictive risk estimates, and eligibility policy. Use approved rules or a clearly labeled synthetic policy service to produce a simulated eligibility outcome. The conversational model must not invent eligibility rules or independently approve credit. Show explanations, uncertainty, and review paths for missing data or borderline cases. No live lending decisions or movement of money is required or authorized by this challenge.

Evaluation evidence

Compare your baseline and proposed system on the same held-out workload. Report the number and mix of cases, label quality, model and prompt versions, and repeated-run variability where relevant. Include failures in the results. If you use a model to judge answers, document its rubric and validate a sample against human or deterministic judgments.

Distinguish these outcomes:

Safe automated resolution:

An eligible case reaches the correct, policy-compliant outcome without human intervention. Report this rate over all in-scope test cases, plus the share of cases on which automation was attempted.

Containment:

A case ends without transfer. Containment alone does not demonstrate that the problem was solved.

Escalation quality:

Cases that require escalation are transferred correctly and include useful handoff context. Report both missed and unnecessary transfers where reference labels permit.

Unsafe outcomes:

Unauthorized disclosures/actions or materially incorrect outcomes, reported with counts and denominators. Zero observed failures in a small test set does not establish zero risk.

Operating efficiency:

End-to-end p50/p95 latency and cost per attempted case and per successful automated resolution. State the workload, sample size, and cost assumptions; use “not defined” when there are no successful resolutions.

Compare relevant service outcomes by language and authorized customer segments, state small-sample limitations, and investigate disparities. Label offline measurements, simulations, and projected business savings separately. Do not describe an offline comparison as a measured production improvement.